



Fundamentos de Internet das Coisas

UNIDADE 03

Componentes fundamentais de sistemas operacionais

A computação utiliza o hardware para processar dados de forma rápida e eficiente, por meio de software. Os computadores consistem em componentes eletrônicos conectados que recebem e processam dados. Programas de computador não podem se comunicar diretamente com os componentes físicos do computador por razões de segurança, o que torna necessário um meio de conexão entre o software e o hardware. Essa tarefa é realizada pelos sistemas operacionais, que gerenciam a interação do usuário com o sistema e permitem que as instruções sejam transmitidas aos dispositivos. Será a partir desse conhecimento que exploraremos mais o nosso ambiente de desenvolvimento Linux recém-criado, para que possamos ter aplicações mais avançadas. Nesta semana, você aprenderá sobre os conceitos fundamentais dos sistemas operacionais, características importantes no uso de computadores e na internet das coisas.

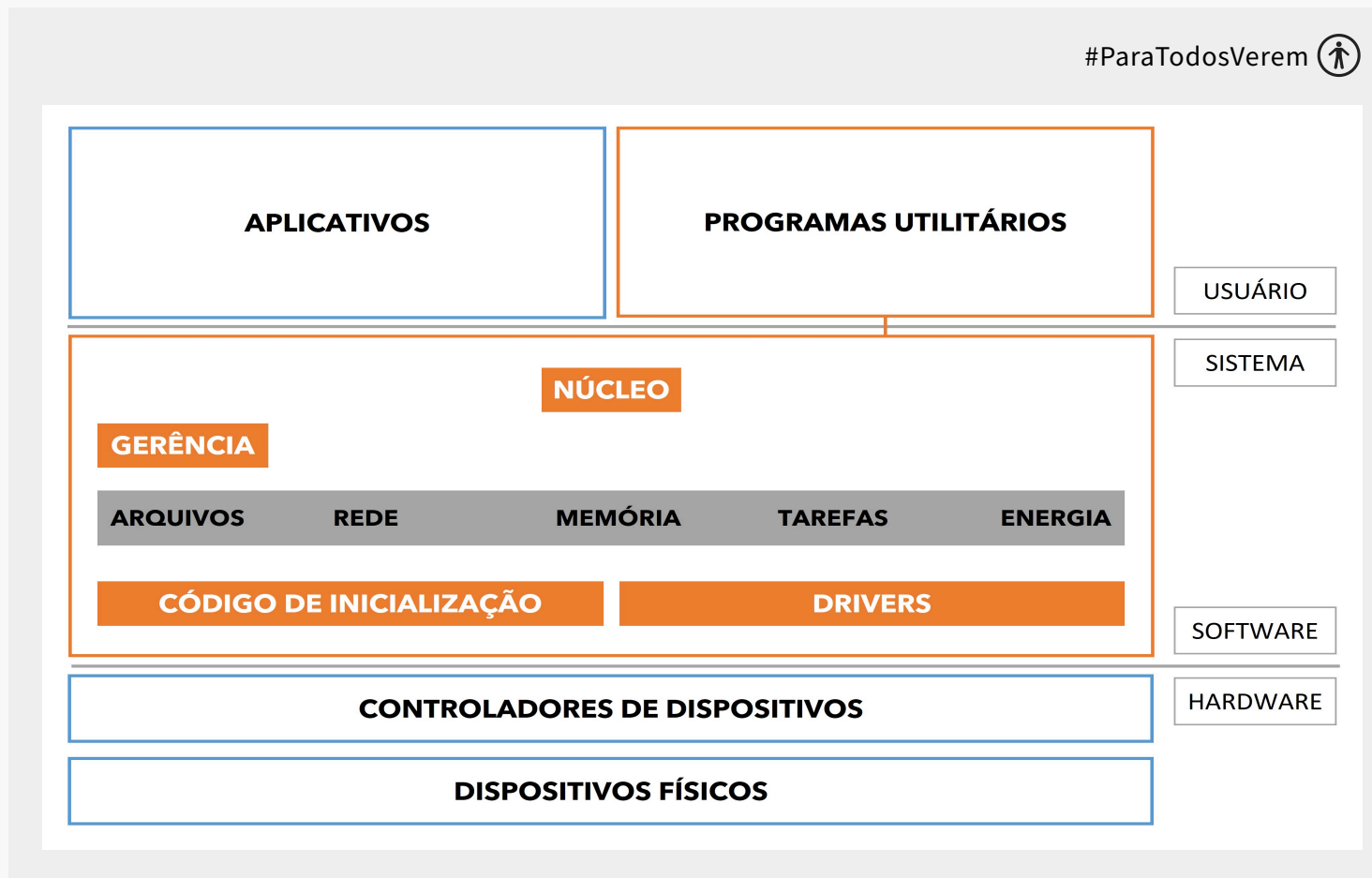
A camada de *software* que atua entre o *hardware* e os programas é chamada sistema operacional, cuja estrutura envolve apresentações de alto e baixo nível. A abstração de recursos utilizada define simplicidade na utilização de interfaces de baixo nível, torna os aplicativos independentes do *hardware* e concede acesso homogêneo a dispositivos com diferentes tecnologias. Ademais, ela apresenta a gerência de recursos como opção de coordenar o uso dos recursos pelos programas, permite o uso compartilhado do processador, além de fatiar o acesso a determinados recursos, como o monitor ou outro dispositivo de saída, por exemplo. Sendo assim, a gerência de recursos possui uma abrangência sobre várias áreas do sistema de computação, como processamento, memória, dispositivos, arquivos e definição das regras de acesso para os recursos disponíveis; dentre as competências principais de um sistema operacional, se destacam as seguintes gerências: dispositivos, processador, memória, arquivos e proteção.

A evolução dos sistemas operacionais acompanhou o início da arquitetura e organização de computadores e ainda ocorre, tal como de sistemas clássicos, como os monolíticos, até os virtualizados. Para a utilização geral no contexto da computação, é preciso identificar os elementos de maior importância ao descrever a arquitetura de um sistema operacional, sendo eles: núcleo, código de *boot* (inicialização), programas utilitários e *drivers*.

A seguir, a Figura 1 representa a estrutura de um sistema operacional típico. Possui em sua base dispositivos físicos e, acima dela, controladores de dispositivos, compondo a camada de *hardware*. Mais acima, temos a camada de *software* e sistema, composta pelo núcleo, com a gerência de: arquivos, rede, memória, proteção, tarefas e energia. Nesse mesmo núcleo, o código de inicialização e os *drivers* dividem a operação com o

núcleo. Ligados a este, na camada de usuário, os programas utilitários dividem a ação do núcleo, além de dividir a camada com os aplicativos.

Figura 1: Estrutura de sistema operacional típico.



Fonte: Adaptado de Maziero (2023).

Outra estrutura de sistema operacional é o Android, que, diferentemente da estrutura típica, apresenta foco na construção de aplicações; com isso, suas camadas são voltadas para serviços e bibliotecas ao usuário.

A plataforma Android é uma pilha de *software* de código aberto baseada no Linux, projetada para operar em diversos dispositivos e formatos. A pilha de *software* do Android é organizada hierarquicamente e permeia funções com: aplicativos de sistema, estrutura da Java API, bibliotecas C e C++ nativas e Android Runtime, camada de abstração de *hardware* (HAL) e, finalmente, o Kernel do Linux.

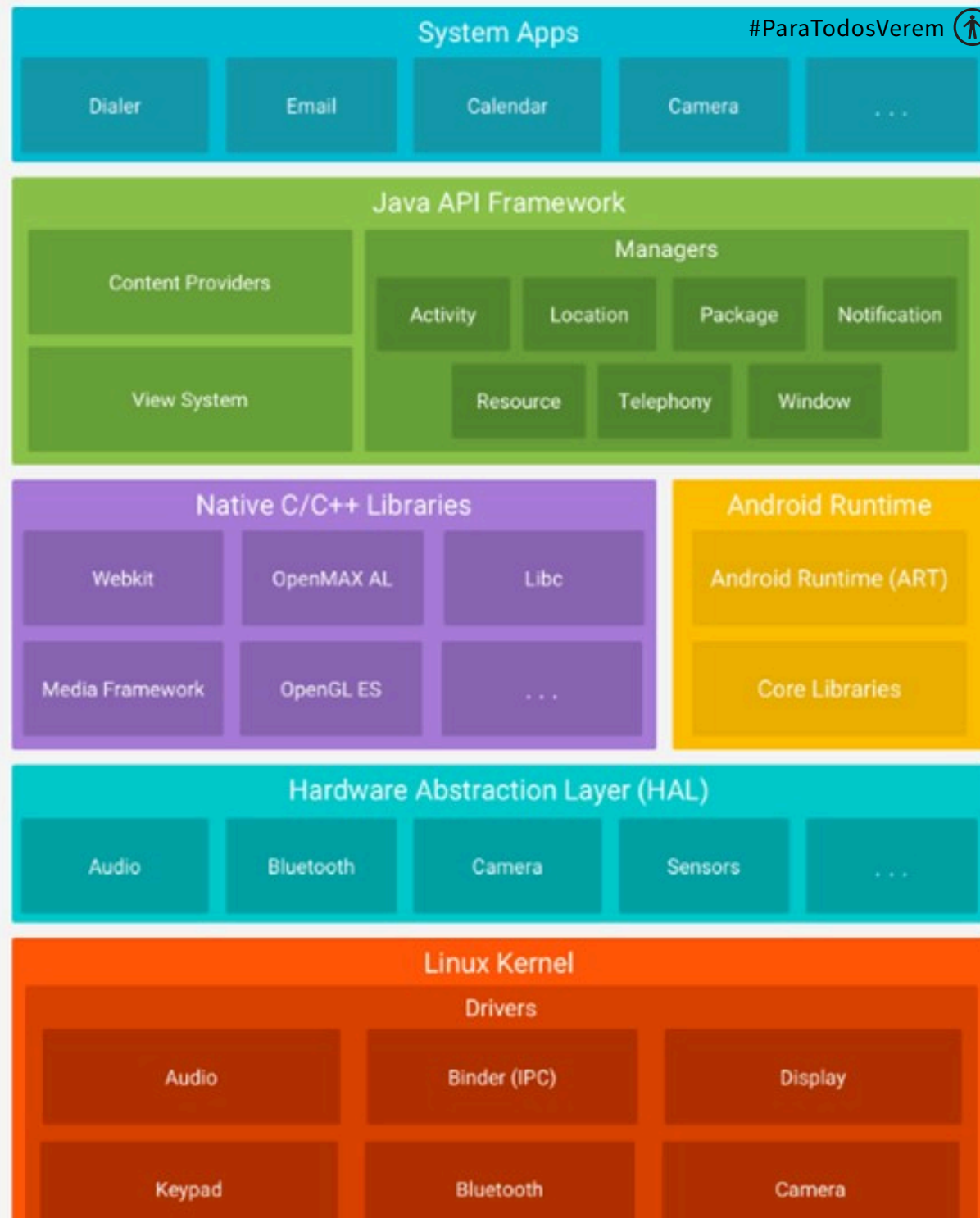


SAIBA MAIS

Acesse <https://developer.android.com/guide/platform?hl=pt-br> e obtenha informações sobre essa arquitetura da plataforma.

A seguir, a Figura 2 representa a estrutura de um sistema operacional Android, baseado em Linux. A representação da estrutura segue a forma empilhada, apresentado, de baixo para cima: Kernel Linux, HAL e bibliotecas C e C++, dividindo o nível da pilha com o Android Runtime. No nível superior, temos Java API e os *systems apps* no topo.

Figura 2: Estrutura do sistema operacional Android.





Fonte: Developers (2023).

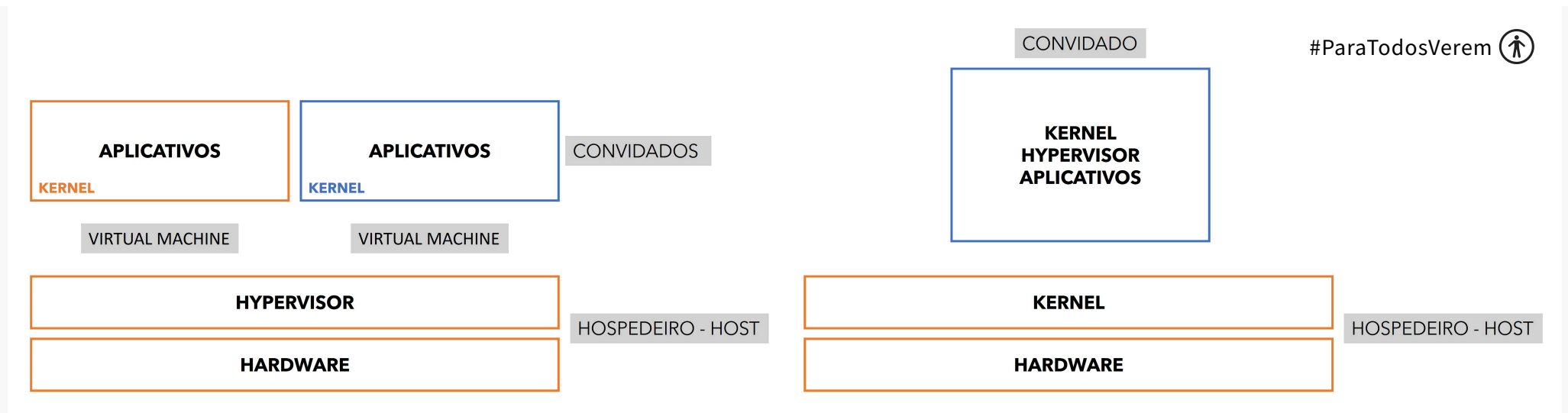
Diversas sugestões de organização dos elementos do sistema operacional surgiram recentemente, buscando atender a necessidades específicas de aplicação, como em ambientes de nuvem computacional. Essas novas abordagens vão além das arquiteturas tradicionais, como a monolítica, em camadas e micronúcleo.

Uma máquina virtual é um *software* que atua como uma camada intermediária, possibilitando a transformação de um sistema em outro. Pela utilização dos serviços disponibilizados pelo sistema operacional (ou *hardware*), é construída uma interface para o outro sistema.

Um ambiente de máquinas virtuais consiste em três partes básicas: sistema real (hospedeiro), camada de virtualização (hipervisor) e sistema virtual (convidado). Dentre os diferentes hipervisores, o do tipo sistema oferece os subtipos: nativo e convidado. O nativo provém da virtualização dos recursos do *hardware* e possibilita construir máquinas virtuais, que executam diversos sistemas operacionais e suas respectivas aplicações. Essa virtualização ocorre diretamente no *hardware*. Já o convidado é realizado em um sistema hospedeiro.

A seguir, a Figura 3 representa o hipervisor nativo à esquerda e o hipervisor convidado à direita. O nativo é representado de baixo para cima com: *hardware* como o *host* e acima o hipervisor, que mantém várias máquinas virtuais com os respectivos sistemas operacionais e suas aplicações (*guests*). Já o hipervisor convidado é representado de baixo para cima com: *hardware* e Kernel como o *host* ou o sistema operacional hospedeiro.

Figura 3: Ambientes de virtualização nativo e convidado.

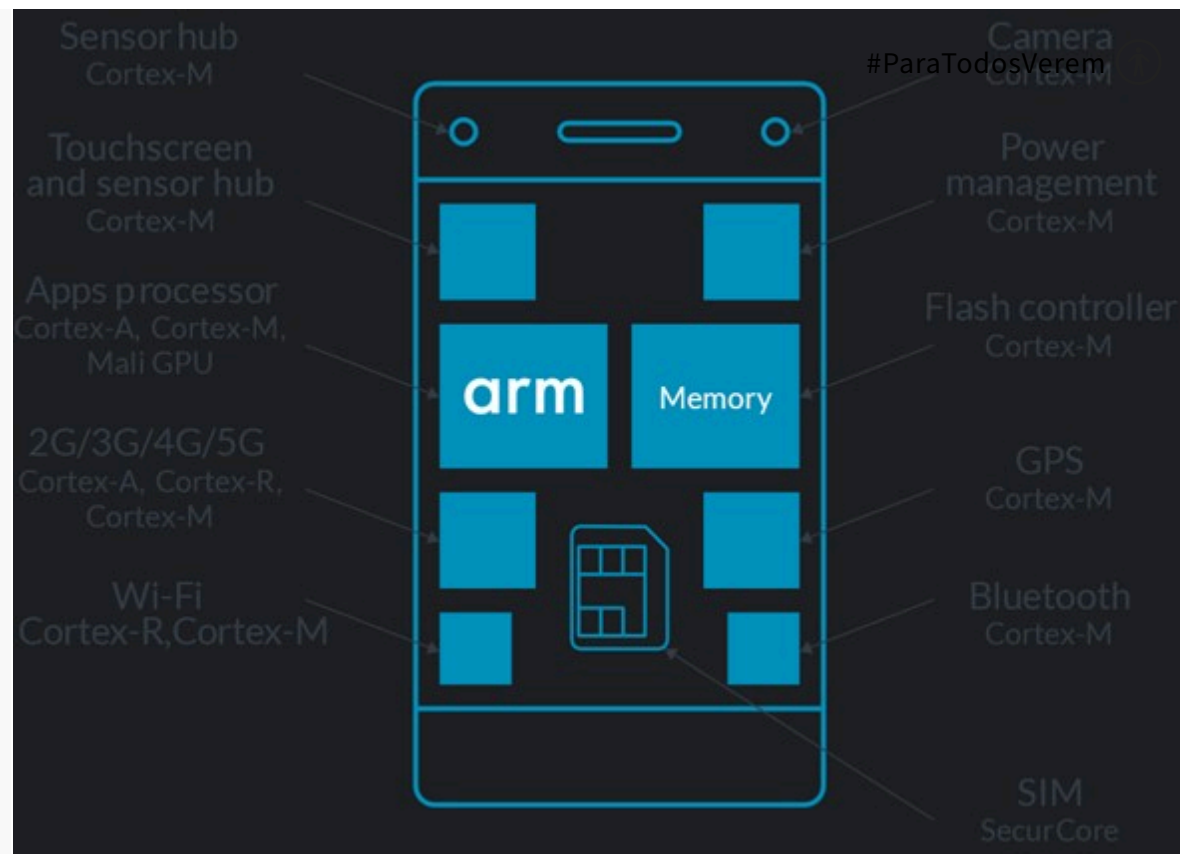


Fonte: Adaptado de Maziero (2023).

Como exemplos de hipervisor convidado, temos o Oracle VirtualBox, VMWare Player, entre outros. Utilizamos a virtualização hipervisor de sistema convidado na construção de nosso ambiente de desenvolvimento, com o Oracle Virtualbox. Já como exemplo de hipervisor nativo, temos o ARM – as Figuras 4 e 5 demonstram, respectivamente, o diagrama e a arquitetura desse processador. O ARM é um processador com arquitetura de 64 bits, projetado para suportar dispositivos móveis e sistemas embarcados. Ele é capaz de executar sistemas operacionais virtualizados, sendo considerado um hipervisor nativo.

A Figura 4 apresenta a composição de um dispositivo Android, que inclui o processador ARM, juntamente a outros módulos como Wi-Fi, 2G, 3G, 4G e 5G, *touchscreen*, sensor de aproximação, câmera, gerenciamento de energia, memória, GPS, Bluetooth e *slot* SIM. Com a capacidade de executar sistemas operacionais virtualizados, o ARMv8 pode fornecer uma plataforma mais flexível e escalável para a execução de aplicativos em dispositivos móveis e sistemas embarcados.

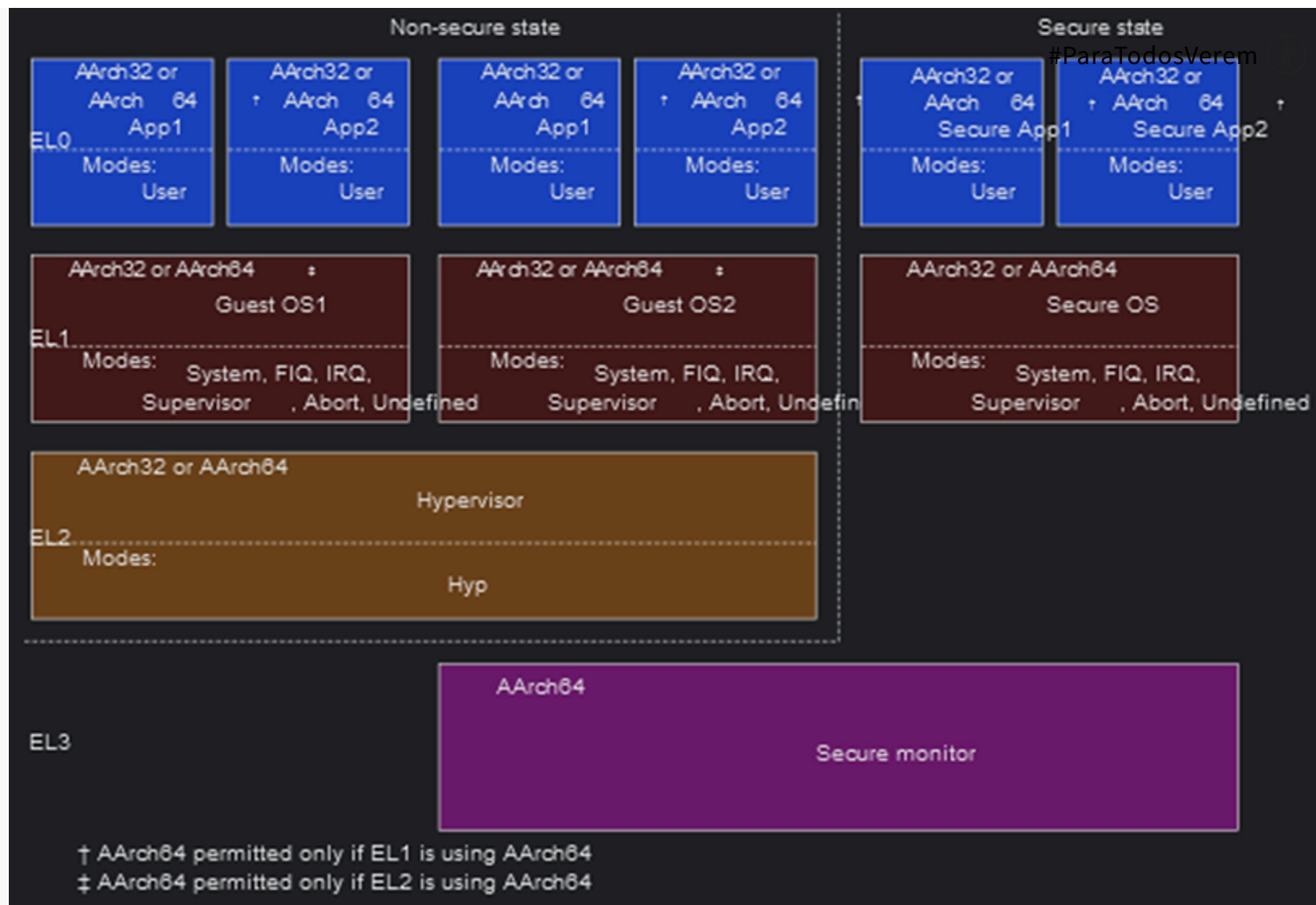
Figura 4: ARM.



Fonte: ARM Developer (2023a).

A Figura 5 representa a arquitetura do ARM: Secure Model, implementada em processador ARMv8. Nela, temos a arquitetura do modelo de segurança descrita de baixo para cima, com o *secure monitor* (camada de monitoramento) acima o hipervisor e, no topo, os níveis de execução particulares deste modelo.

Figura 5: Arquitetura ARMv8.



Fonte: ARM Developer (2023b)

Existem outras maneiras de isolar aplicativos ou subsistemas em um sistema operacional, além da virtualização. Uma delas é a virtualização do espaço do usuário (*userspace*). Nessa abordagem, conhecida como servidores virtuais ou contêineres, o espaço do usuário do sistema operacional é separado em áreas isoladas, chamadas domínios ou contêineres. Cada domínio recebe uma porção dos recursos do sistema operacional, como memória, tempo de processador e espaço em disco. Plataformas de gerenciamento de contêineres, como Docker, Singularity, Kubernetes e Linux Containers (LXC/LXC), são exemplos de mecanismos de isolamento de recursos disponíveis no núcleo Linux.

Você sabia que a união dos temas de abstração de recursos, virtualização e Linux é extremamente importante para a computação moderna?

Bom, a abstração de recursos permite que as aplicações sejam independentes do *hardware* subjacente, o que significa que elas podem ser executadas em diferentes plataformas sem problemas de compatibilidade. Já a virtualização permite criar múltiplas máquinas virtuais em um único servidor físico, aumentando a eficiência e utilização de recursos. O Linux é um sistema operacional altamente flexível e personalizável, amplamente utilizado em ambientes de virtualização e nuvens públicas e privadas.

A união desses temas é fundamental para a criação de infraestruturas de TI escaláveis, seguras e eficientes, que são essenciais para a maioria das empresas modernas. Além disso, esses conceitos são cruciais para o desenvolvimento de sistemas embarcados e sua implementação no contexto da internet das coisas.

Nós podemos começar explorando os conceitos fundamentais de sistemas operacionais, abrangendo a abstração de recursos, virtualização e execução no ambiente Linux. Podemos utilizar o Linux como cenário e como um importante ator desses preceitos, por meio do uso de uma das formas de sistema operacional virtualizado, o Linux convidado. Durante a prática, é possível demonstrar os fundamentos da abstração de recursos, a criação de máquinas virtuais e sua implementação no Linux.

| Conclusão

Durante esta semana, foram abordados os conceitos fundamentais de sistemas operacionais, com ênfase nas estruturas principais. A compreensão desses conceitos será essencial para o planejamento e implementação de futuros projetos de sistemas embarcados voltados para a internet das coisas.

| Referências Bibliográficas

ARM DEVELOPER. **Learn the architecture:** introducing the Arm architecture. Disponível em:
<https://developer.arm.com/documentation/102404/0201/About-the-Arm-architecture>. Acesso em 1 mar. 2023a.

ARM DEVELOPER. **Arm Cortex-A53 MPCore Processor Technical Reference Manual r0p4**. Disponível em: <https://developer.arm.com/documentation/ddi0500/j/Programmers-Model/ARMv8-A-architecture-concepts/ARMv8-security-model?lang=en>. Acesso em 1 mar. 2023b.

DEVELOPERS. **Arquitetura da plataforma**. Disponível em: <https://developer.android.com/guide/platform?hl=pt-br>. Acesso em: 1 mar. 2023.

MAZIERO, C. **Sistemas operacionais: conceitos e mecanismos**. Curitiba: Editora da UFPR, 2019.

TANENBAUM, A. S. **Sistemas operacionais modernos**. 4. ed. São Paulo: Pearson Education do Brasil, 2016.

